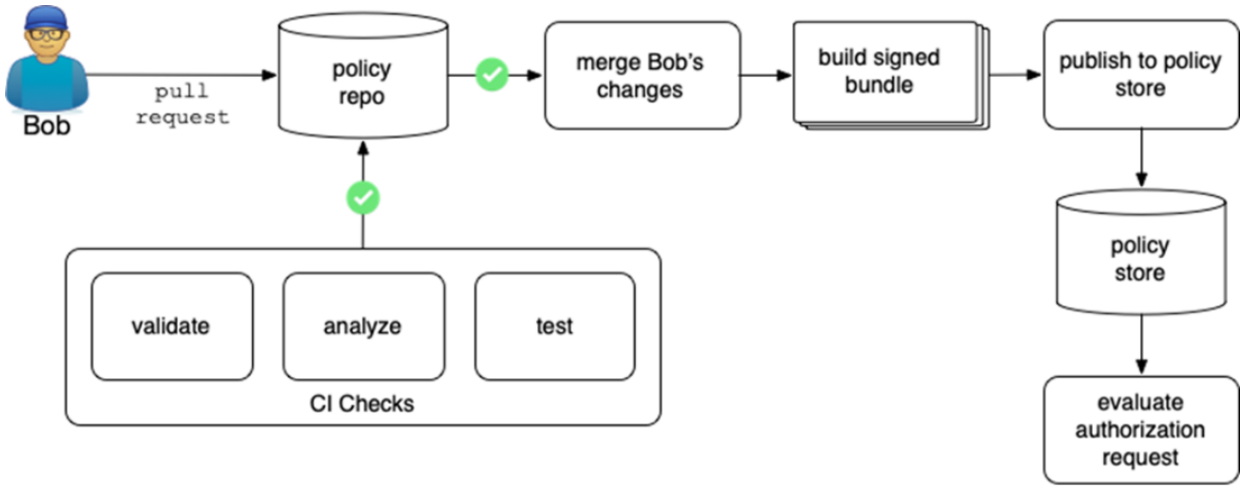


Figure 10.6 Continuous integration for policies. Bob's pull request triggers validation, analysis, and testing. After the checks pass, his changes are merged, bundled into a signed package, and published to a policy store, where the authorizer uses them to evaluate requests at runtime.

This pipeline provided ACME with two key benefits. First, it identified issues early: Misspelled action names or references to non-existent attributes were flagged before they could cause runtime failures. Second, it established a repeatable process: every bundle was built consistently, with the same checks, and only from reviewed code. Authorization transitioned from being fragile "tribal knowledge" to becoming a formal part of the company's engineering discipline.

10.3 Testing policies effectively

Even the most carefully written Cedar policies can miss subtle issues if they aren't tested against real-world scenarios. Authorization logic runs for every access request. Simply verifying that policies compile isn't enough; you need confidence that they consistently enforce the rules you intend.

At ACME, the identity team learned this lesson the hard way. A change to project policies once prevented managers of owners from editing documents, an edge case no one had thought to test. The result was several days of confusion until engineers identified the gap. Afterward, the team adopted a discipline of policy testing alongside the Customer Collaboration platform. Every policy change now undergoes a set of tests before deployment, just like application logic.